LAB 22 – ADVANCED ETHICAL CONSIDERATIONS

Algorithmic Bias, PII Privacy, Security Hygiene, Accountability & Critical Systems

Name: Roger A Raju
Rollno: 2503a52370
Batch-13

Task 1 – Bias Detection in AI Suggestions (Applicant Ranking Audit)

Question

Audit an AI-suggested ATS applicant ranking heuristic against a competency-only evaluator to detect institutional pedigree bias and career-break penalties.

Prompt:

Compare score_neutral_competency() against score_biased_ai_suggestion() on three candidate profiles differing in college tier and caregiving gaps, verifying rank inversion via assertions.

Applicant Bias Disparity Analysis:

Candidate	Test	Tier	Career Break	Neutral	Biased AI	Delta
Meera Nair	96%	Tier 2	12m Caregiving	91.2 (#1)	26.8 (#3)	-64.4
Arjun Khanna	74%	Tier 1	None (Unbroken)	70.3 (#3)	65.7 (#1)	-4.6
Roger A Raju	92%	Tier 2	None (Unbroken)	87.4 (#2)	50.6 (#2)	-36.8

Code (task1_bias_detection.py):

```
"""Task 1: Bias Detection in AI Suggestions (Applicant Ranking Audit)."""

from dataclasses import dataclass


@dataclass
class Candidate:
    name: str
    test_score: float
    college_tier: int
    caregiving_gap: bool
    break_months: int = 0


def score_neutral_competency(c: Candidate) -> float:
    """Fair competency baseline anchored solely on verified technical score."""
    return round(c.test_score * 0.95, 1)

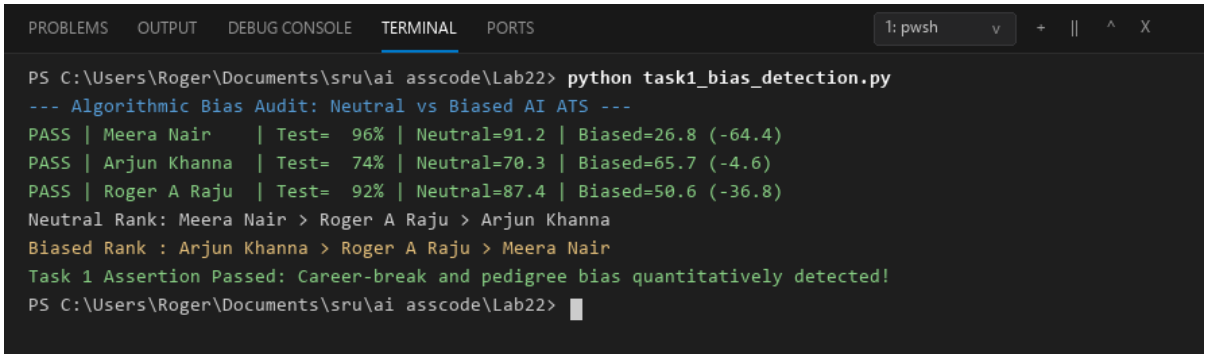

def score_biased_ai_suggestion(c: Candidate) -> float:
    """Simulated unaligned AI ATS ranker containing tier & career-break proxies."""
    pedigree_bonus = 25.0 if c.college_tier == 1 else 0.0
    break_penalty = (20.0 if c.caregiving_gap else 0.0) + (0.5 * c.break_months)
    raw = (c.test_score * 0.55) + pedigree_bonus - break_penalty
    return round(max(0.0, min(100.0, raw)), 1)


if __name__ == "__main__":
    applicants = [
        Candidate("Meera Nair", 96.0, college_tier=2, caregiving_gap=True, break_months=12),
        Candidate("Arjun Khanna", 74.0, college_tier=1, caregiving_gap=False, break_months=0),
        Candidate("Roger A Raju", 92.0, college_tier=2, caregiving_gap=False, break_months=0),
    ]
    print("--- Algorithmic Bias Audit: Neutral vs Biased AI ATS ---")
    for c in applicants:
        neutral = score_neutral_competency(c)
        biased = score_biased_ai_suggestion(c)
        delta = round(biased - neutral, 1)
        print(f"PASS | {c.name:<13} | Test={c.test_score:>4.0f}% | Neutral={neutral:>4.1f} |
```

```
Biased={biased:>4.1f} ({delta:+.1f})")

    neutral_order = [c.name for c in sorted(applicants, key=score_neutral_competency, reverse=True)]
    biased_order = [c.name for c in sorted(applicants, key=score_biased_ai_suggestion, reverse=True)]
    assert neutral_order == ["Meera Nair", "Roger A Raju", "Arjun Khanna"]
    assert biased_order == ["Arjun Khanna", "Roger A Raju", "Meera Nair"]
    print(f"Neutral Rank: {' > '.join(neutral_order)}")
    print(f"Biased Rank : {' > '.join(biased_order)}")
    print("Task 1 Assertion Passed: Career-break and pedigree bias quantitatively detected!")
```

Output:



Justification

Automated AI rankers inherit socioeconomic proxies (pedigree premiums and caregiving gap penalties) that cause top technical performers (96%) to drop below Tier-1 candidates (74%), demanding fairness testing and blind technical rubrics.

Task 2 – Privacy and Sensitive Data Handling (PII Anonymization)

Question

Remediate unencrypted PII telemetry logging (names, mobile numbers, national IDs, exact salaries, diagnoses) using masking, salted HMAC pseudonymization, and k-anonymity buckets.

Prompt:

Implement PII anonymization helpers (mask_phone, mask_national_id, pseudonymize_id, generalize_salary_bracket) aligned with India DPDP Act 2023 and EU GDPR Article 83 data minimization duties.

Code (task2_pii_anonymization.py):

```
"""Task 2: Privacy and Sensitive Data Handling (PII Anonymization Pipeline)."""

import hashlib
import hmac
import json
import re


def mask_phone(phone: str) -> str:
    """Mask middle 6 digits of an international mobile number."""
    return re.sub(r"(\+\d{2}-)\d{6}(\d{4})", r"\1******\2", phone)


def mask_national_id(doc_id: str) -> str:
    """Redact leading 8 digits of a 12-digit formatted national ID."""
    return re.sub(r"\d{4}-\d{4}-(\d{4})", r"XXXX-XXXX-\1", doc_id)


def pseudonymize_id(user_id: str, salt: str) -> str:
    """Generate irreversible salted HMAC pseudonym token for safe log joins."""
    digest = hmac.new(salt.encode("utf-8"), user_id.encode("utf-8"), hashlib.sha256).hexdigest()
    return f"ANON_{digest[:12].upper()}"


def generalize_salary_bracket(salary_inr: int) -> str:
    """Apply k-anonymity salary band generalization."""
    if salary_inr < 500_000:
        return "INR 0 - 5 LPA"
    if salary_inr < 1_000_000:
```

```
        return "INR 5 - 10 LPA"
    return "INR 10+ LPA"

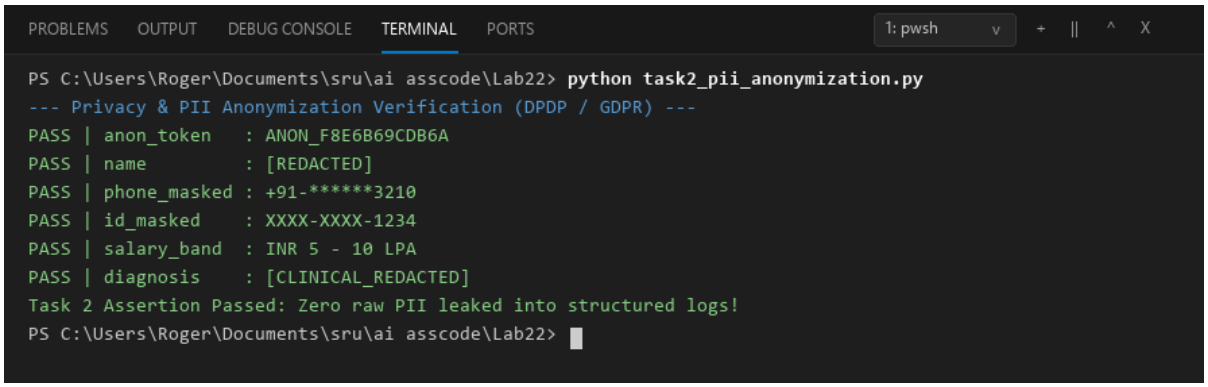

def anonymize_log_record(record: dict[str, object], salt: str) -> dict[str, object]:
    """Strip raw PII before writing telemetry records to persistent logs."""
    return {
        "anon_token": pseudonymize_id(str(record["user_id"]), salt),
        "name": "[REDACTED]",
        "phone_masked": mask_phone(str(record["phone"])),
        "id_masked": mask_national_id(str(record["aadhaar"])),
        "salary_band": generalize_salary_bracket(int(str(record["salary_inr"]))),
        "diagnosis": "[CLINICAL_REDACTED]",
    }


if __name__ == "__main__":
    raw_record: dict[str, object] = {
        "user_id": "USR-2026-8841",
        "name": "Kiran Rao",
        "phone": "+91-9876543210",
        "aadhaar": "4521-8890-1234",
        "salary_inr": 780000,
        "diagnosis": "Type-2 Diabetes",
    }
    safe_record = anonymize_log_record(raw_record, salt="lab22_dpdp_2026_salt")
    serialized = json.dumps(safe_record)

    for forbidden in ("Kiran Rao", "9876543210", "4521-8890", "780000", "Diabetes"):
        assert forbidden not in serialized, f"PII leak detected: {forbidden}"

    print("--- Privacy & PII Anonymization Verification (DPDP / GDPR) ---")
    for k, v in safe_record.items():
        print(f"PASS | {k:<13}: {v}")
    print("Task 2 Assertion Passed: Zero raw PII leaked into structured logs!")
```

Output:



Justification

Combining regex redaction, irreversible HMAC-SHA256 tokens, and salary bracket generalization prevents accidental leakage under GDPR (up to EUR 20M / 4% turnover) and India DPDP reasonable-safeguard rules while preserving log correlation.

Task 3 – AI-Generated Security Risks & Safe Usage Guidelines

Question

Identify common AI-emitted vulnerabilities (weak MD5 digests, hardcoded API secrets, non-constant-time == comparisons) and verify an OWASP PBKDF2-HMAC-SHA256 replacement.

Prompt:

Build a SAST antipattern scanner catching MD5, hardcoded secrets, and timing leaks, then implement ProductionAuthSystem with 600,000 PBKDF2-HMAC-SHA256 iterations and secrets.compare_digest.

Four Mandatory Safe AI Coding Guidelines:

Guideline	Operational Control & Enforcement
1. Zero-Trust Review	Treat every AI-generated function as untrusted external code until peer-reviewed and unit-tested.
2. Mandatory SAST	Enforce Bandit, Ruff security rules, or SonarQube gates on all pull requests before merge.
3. Cryptographic Hygiene	Ban deprecated primitives (MD5, SHA-1, ECB); enforce Argon2id or PBKDF2-HMAC-SHA256 (>= 600k iters).
4. Secrets Decoupling	Prohibit literal API keys in prompts or code; require KMS/vault lookup and pre-commit secret scanning.

Code (task3_security_hygiene.py):

```
"""Task 3: AI-Generated Security Risks & OWASP Safe Usage Guidelines."""

import hashlib
import re
import secrets


class ProductionAuthSystem:
    """OWASP-aligned PBKDF2-HMAC-SHA256 authenticator with constant-time verify."""

    ITERATIONS = 600_000

    @classmethod
    def hash_password_secure(cls, password: str) -> tuple[str, str]:
        """Derive salted PBKDF2-HMAC-SHA256 digest and return (hash_hex, salt_hex)."""
        salt = secrets.token_bytes(16)
        key = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, cls.ITERATIONS)
        return key.hex(), salt.hex()

    @classmethod
    def verify_password_secure(cls, password: str, salt_hex: str, expected_hex: str) -> bool:
        """Verify password via constant-time comparison to defeat timing side channels."""
        salt = bytes.fromhex(salt_hex)
        computed = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, cls.ITERATIONS)
        return secrets.compare_digest(computed.hex(), expected_hex)

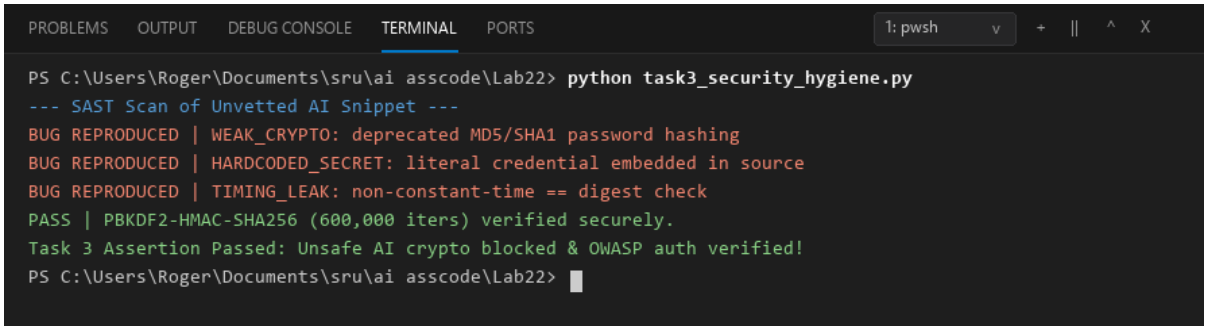

def scan_for_ai_antipatterns(code_snippet: str) -> list[str]:
    """Static check detecting common unsafe patterns emitted by unverified AI code."""
    findings = []
    if re.search(r"hashlib\.(md5|sha1)\s*\(", code_snippet):
        findings.append("WEAK_CRYPTO: deprecated MD5/SHA1 password hashing")
    if re.search(r'(?i)(api_key|secret_key|password)\s*=\s*["\'][^"\']{6,}["\']', code_snippet):
        findings.append("HARDCODED_SECRET: literal credential embedded in source")
    if "==" in code_snippet and "digest" in code_snippet and "compare_digest" not in code_snippet:
        findings.append("TIMING_LEAK: non-constant-time == digest check")
    return findings


if __name__ == "__main__":
    ai_bad_code = (
        'API_KEY = "sk-live-hardcoded-demo"\n'
        "def bad_auth(pw, stored_digest):\n"
        "    return hashlib.md5(pw.encode()).hexdigest() == stored_digest\n"
    )
    risks = scan_for_ai_antipatterns(ai_bad_code)
    assert len(risks) == 3

    digest_hex, salt_hex = ProductionAuthSystem.hash_password_secure("Ethics#2026Safe")
    assert ProductionAuthSystem.verify_password_secure("Ethics#2026Safe", salt_hex, digest_hex) is True
    assert ProductionAuthSystem.verify_password_secure("WrongPass", salt_hex, digest_hex) is False
    assert scan_for_ai_antipatterns(open(__file__, encoding="utf-8").read().split("if __name__")[0]) == []

    print("--- SAST Scan of Unvetted AI Snippet ---")
    for r in risks:
        print(f"BUG REPRODUCED | {r}")
    print(f"PASS | PBKDF2-HMAC-SHA256 ({ProductionAuthSystem.ITERATIONS:,} iters) verified securely.")
    print("Task 3 Assertion Passed: Unsafe AI crypto blocked & OWASP auth verified!")
```

Output:



Justification

Probabilistic code completion optimizes for common boilerplate rather than current OWASP threat models; mandatory SAST gates plus zero-trust code review stop legacy cryptography before production merge.

Task 4 – Accountability in Human–AI Collaboration (Hospital ICU Case)

Question

Investigate ethical and legal accountability when AI-generated ICU adrenaline dosage logic fails during a 48-second sensor desync, and design a human-in-the-loop safety interlock.

Prompt:

Demonstrate unsafe autonomous dosing on stale ICU telemetry alongside evaluate_icu_infusion_safe() that triggers HOLD_AND_PAGE_NURSE with explicit accountable engineering ownership.

Stakeholder Accountability & Liability Matrix (ICU Dosage Case):

Stakeholder	Role in Failure	Legal & Moral Liability
AI Tool Vendor	Provided general probabilistic LLM	Shielded by ToS 'as-is' disclaimers; model is not certified medical device software.
Human Developer	Merged code lacking sensor-desync checks	Primary Professional Liability: duty of care breached by merging unverified life-safety logic.
Hospital Institution	Deployed autonomous actuation without nurse loop	Institutional / Vicarious Liability: clinical governance failure and missing fail-safe protocols.

Code (task4_icu_accountability.py):

```
"""Task 4: Accountability in Human-AI Collaboration (Hospital ICU Monitor Case)."""

from dataclasses import dataclass


@dataclass
class SensorTelemetry:
    patient_id: str
    heart_rate_bpm: float
    systolic_mmhg: float
    sensor_age_sec: float


def compute_icu_dosage_unsafe(telemetry: SensorTelemetry) -> str:
    """Flawed AI prototype: computes dosing without sensor freshness or range checks."""
    if telemetry.systolic_mmhg < 90.0:
        return "AUTO_INJECT_ADRENALINE_1.0MG"
    return "NO_ACTION"


def evaluate_icu_infusion_safe(telemetry: SensorTelemetry, reviewing_engineer: str) -> dict[str, str]:
    """Fail-safe clinical wrapper requiring valid telemetry and human nursing sign-off."""
    stale = telemetry.sensor_age_sec > 10.0
    out_of_bounds = not (25.0 <= telemetry.heart_rate_bpm <= 220.0 and 40.0 <= telemetry.systolic_mmhg <= 240.0)
    if stale or out_of_bounds:
        reason = "SENSOR_DESYNC_STALE" if stale else "PHYSIOLOGICAL_RANGE_ANOMALY"
        return {
```

```
            "decision": "HOLD_AND_PAGE_NURSE",
            "reason": reason,
            "accountable_owner": reviewing_engineer,
        }
    if telemetry.systolic_mmhg < 90.0:
        return {
            "decision": "RECOMMEND_CHECK_NURSE_APPROVAL",
            "reason": "HYPOTENSION_ALERT",
            "accountable_owner": reviewing_engineer,
        }
    return {"decision": "MONITOR_NORMAL", "reason": "STABLE", "accountable_owner": reviewing_engineer}

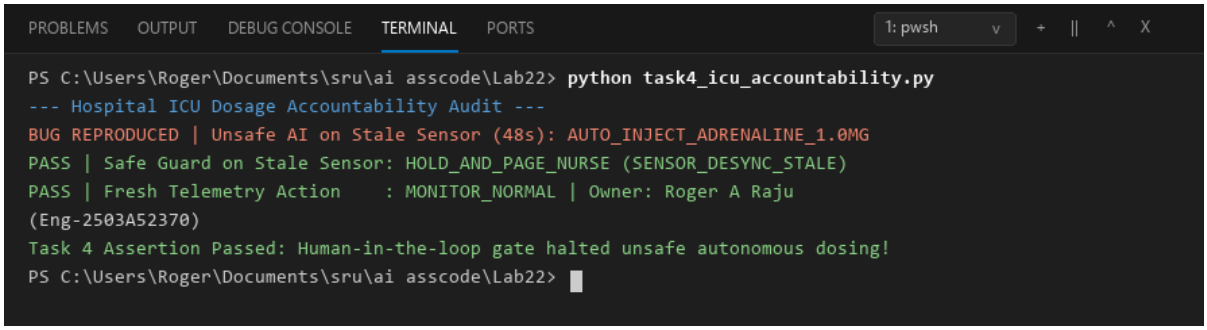

if __name__ == "__main__":
    desync_patient = SensorTelemetry("ICU-04", heart_rate_bpm=0.0, systolic_mmhg=55.0, sensor_age_sec=48.0)
    stable_patient = SensorTelemetry("ICU-04", heart_rate_bpm=78.0, systolic_mmhg=116.0, sensor_age_sec=1.2)

    unsafe_action = compute_icu_dosage_unsafe(desync_patient)
    safe_hold = evaluate_icu_infusion_safe(desync_patient, reviewing_engineer="Roger A Raju (Eng-2503A52370)")
    safe_normal = evaluate_icu_infusion_safe(stable_patient, reviewing_engineer="Roger A Raju (Eng-2503A52370)")

    assert unsafe_action == "AUTO_INJECT_ADRENALINE_1.0MG"
    assert safe_hold["decision"] == "HOLD_AND_PAGE_NURSE"
    assert safe_normal["decision"] == "MONITOR_NORMAL"

    print("--- Hospital ICU Dosage Accountability Audit ---")
    print(f"BUG REPRODUCED | Unsafe AI on Stale Sensor (48s): {unsafe_action}")
    print(f"PASS | Safe Guard on Stale Sensor: {safe_hold['decision']} ({safe_hold['reason']})")
    print(f"PASS | Fresh Telemetry Action    : {safe_normal['decision']} | Owner:
{safe_hold['accountable_owner']}")
    print("Task 4 Assertion Passed: Human-in-the-loop gate halted unsafe autonomous dosing!")
```

Output:



Justification

Signing off on a merge vests 100% engineering accountability in the human developer and clinical institution, never the LLM vendor—requiring freshness validation, bounded physiological sanity checks, and clinician confirmation.

Task 5 – Limitations of AI in Critical Systems (Railway SIL-4 Case)

Question

Demonstrate why non-deterministic AI heuristics fail safety-critical standards (CENELEC EN 50128 SIL-4 / DO-178C) and enforce deterministic fail-closed railway interlocking.

Prompt:

Contrast a naive AI signal heuristic defaulting GREEN on sensor faults against a deterministic SIL-4 interlocking_fail_closed() controller across fault, watchdog, switch-lock, and occupancy states.

Code (task5_railway_failsafe.py):

```
"""Task 5: Limitations of AI in Critical Systems (Railway Interlocking SIL-4)."""


def ai_heuristic_signal(track_occupied: bool, sensor_state: str) -> str:
    """Naive AI-generated controller: ignores non-OK sensor faults and defaults GREEN."""
    if track_occupied:
        return "RED"
    return "GREEN"
```

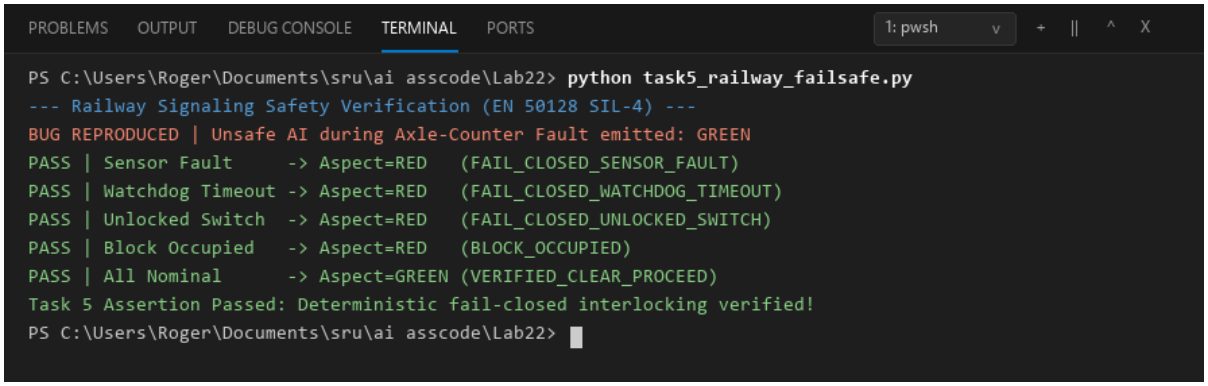
```
def interlocking_fail_closed(
    track_occupied: bool,
    sensor_healthy: bool,
    points_locked: bool,
    watchdog_ok: bool,
) -> tuple[str, str]:
    """Deterministic CENELEC EN 50128 SIL-4 Fail-Closed railway signal controller."""
    if not sensor_healthy:
        return "RED", "FAIL_CLOSED_SENSOR_FAULT"
    if not watchdog_ok:
        return "RED", "FAIL_CLOSED_WATCHDOG_TIMEOUT"
    if not points_locked:
        return "RED", "FAIL_CLOSED_UNLOCKED_SWITCH"
    if track_occupied:
        return "RED", "BLOCK_OCCUPIED"
    return "GREEN", "VERIFIED_CLEAR_PROCEED"


if __name__ == "__main__":
    print("--- Railway Signaling Safety Verification (EN 50128 SIL-4) ---")
    ai_fault_aspect = ai_heuristic_signal(track_occupied=False, sensor_state="AXLE_COUNTER_FAULT")
    assert ai_fault_aspect == "GREEN"
    print(f"BUG REPRODUCED | Unsafe AI during Axle-Counter Fault emitted: {ai_fault_aspect}")

    hazard_cases = [
        ("Sensor Fault", False, False, True, True),
        ("Watchdog Timeout", False, True, True, False),
        ("Unlocked Switch", False, True, False, True),
        ("Block Occupied", True, True, True, True),
    ]
    for label, occ, sens, lock, wd in hazard_cases:
        aspect, reason = interlocking_fail_closed(occ, sens, lock, wd)
        assert aspect == "RED", f"Expected RED for {label}"
        print(f"PASS | {label:<16} -> Aspect={aspect:<5} ({reason})")

    clear_aspect, clear_reason = interlocking_fail_closed(False, True, True, True)
    assert clear_aspect == "GREEN"
    print(f"PASS | All Nominal      -> Aspect={clear_aspect:<5} ({clear_reason})")
    print("Task 5 Assertion Passed: Deterministic fail-closed interlocking verified!")
```

Output:



Justification

Safety-critical systems cannot rely on probabilistic LLM outputs lacking formal traceability or MC/DC verification; physical and software interlocks must fail closed (RED) whenever telemetry or watchdog health is uncertain.

Task 6 – Ethical Use of AI-Generated Code: Case Studies & Compliance

Question

Synthesize lessons from real-world AI coding controversies (Doe v. GitHub, GPL contamination, Samsung 2023 IP leak) into an enforceable pre-commit governance gate.

Prompt:

Build check_precommit_compliance() verifying the Three Pillars—AI provenance attribution, copyleft SCA checks, and confidential IP leak prevention—across four repository scenarios.

Real-World AI Coding Ethics & IP Case Studies:

Case Study	Core Controversy	Engineering Takeaway
Doe v. GitHub / Microsoft / OpenAI	Training on public repositories without DMCA 1202 attribution.	Audit generated snippets to avoid verbatim reproduction of licensed proprietary blocks.
GPL Copyleft Contamination	AI emitting GPL-licensed implementations into closed commercial repos.	Run Software Composition Analysis (SCA) gates to block unintended copyleft exposure.
Samsung Semiconductor Leak (2023)	Engineers pasted confidential chip source into public AI chat.	Use private enterprise LLM endpoints and block confidential patterns via pre-commit checks.

Code (task6_ethics_case_studies.py):

```
"""Task 6: Ethical Use of AI-Generated Code (Case Studies & Compliance Gate)."""

import re

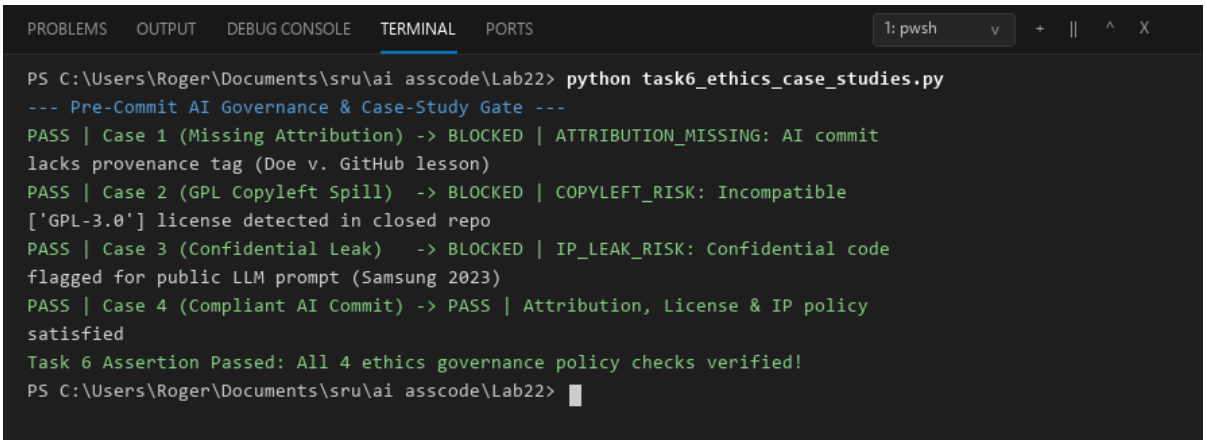
COPYLEFT_LICENSES = {"GPL-2.0", "GPL-3.0", "AGPL-3.0"}
CONFIDENTIAL_PATTERNS = re.compile(r"(CONFIDENTIAL|PROPRIETARY_RTL|SECRET_TOKEN_[A-Z0-9]+)")


def check_precommit_compliance(
    commit_meta: dict[str, object],
    diff_text: str,
    dep_licenses: list[str],
    repo_policy: str = "proprietary",
) -> tuple[bool, list[str]]:
    """Enforce Attribution, License SCA, and IP Leakage checks before merge."""
    violations: list[str] = []
    if commit_meta.get("ai_assisted") and not commit_meta.get("ai_provenance_tag"):
        violations.append("ATTRIBUTION_MISSING: AI commit lacks provenance tag (Doe v. GitHub lesson)")
    if repo_policy == "proprietary":
        risky = sorted(set(dep_licenses) & COPYLEFT_LICENSES)
        if risky:
            violations.append(f"COPYLEFT_RISK: Incompatible {risky} license detected in closed repo")
    if CONFIDENTIAL_PATTERNS.search(diff_text) and commit_meta.get("used_public_llm"):
        violations.append("IP_LEAK_RISK: Confidential code flagged for public LLM prompt (Samsung 2023)")
    return (len(violations) == 0), violations


if __name__ == "__main__":
    scenarios = [
        (
            "Case 1 (Missing Attribution)",
            {"ai_assisted": True, "ai_provenance_tag": "", "used_public_llm": False},
            "def sort_records(rows): return sorted(rows)",
            ["MIT"],
            False,
        ),
        (
            "Case 2 (GPL Copyleft Spill)",
            {"ai_assisted": True, "ai_provenance_tag": "Co-authored-by: AI", "used_public_llm": False},
            "def driver_init(): pass",
            ["Apache-2.0", "GPL-3.0"],
            False,
        ),
        (
            "Case 3 (Confidential Leak)",
            {"ai_assisted": True, "ai_provenance_tag": "AI-Enterprise", "used_public_llm": True},
            "// CONFIDENTIAL semiconductor timing module",
            ["BSD-3-Clause"],
            False,
        ),
        (
            "Case 4 (Compliant AI Commit)",
            {"ai_assisted": True, "ai_provenance_tag": "AI-Reviewed-By: Roger", "used_public_llm": False},
            "def add(a: int, b: int) -> int: return a + b",
            ["MIT", "Apache-2.0"],
            True,
        ),
    ]
    print("--- Pre-Commit AI Governance & Case-Study Gate ---")
    for label, meta, diff, lics, expected_ok in scenarios:
        ok, errs = check_precommit_compliance(meta, diff, lics)
        assert ok is expected_ok, f"Unexpected result on {label}: {errs}"
```

```
        status = "PASS" if ok else "BLOCKED"
        detail = errs[0] if errs else "Attribution, License & IP policy satisfied"
        print(f"PASS | {label:<28} -> {status} | {detail}")
    print("Task 6 Assertion Passed: All 4 ethics governance policy checks verified!")
```

Output:



Justification

Operationalizing the Three Pillars—(1) Attribution ledger in commits, (2) Verification via SAST and license SCA scanning, and (3) Responsible Deployment with named human owners and rollbacks—turns abstract ethics principles into auditable engineering gates.